Vrije Universiteit Amsterdam

Bachelor Thesis

Stock Price Prediction Using Machine Learning Methods

Author: Zeynep Uzun (2733079)

supervisor: Wan Fokkink
2nd reader: Vincent Francois Lavet

A thesis submitted in fulfillment of the requirements for the VU Bachelor of Science degree in Computer Science July 2, 2024

Contents

1 Introduction **3**

2 Literature Review **5**
2.1 Single Model Approaches 5
2.2 Hybrid Model Approaches 6
2.3 Comparative Studies 6

3 Machine Learning Models **7**
3.1 Overview of Recurrent Neural Networks (RNN's) 7
3.2 Overview of Long Short-Term Memory (LSTM) 11
3.3 Overview of Support Vector Machine (SVM) 15
3.4 Support Vector Regression (SVR) 19

4 Dataset **21**

5 Methodology **22**
5.1 Exploratory Data Analysis (EDA) 22
5.2 Features 23
5.3 Experimental Setup 24
5.3.1 Data Preprocessing 24
5.3.2 Data Splitting 25
5.3.3 Phase 1: Hyperparameter Tuning 25
5.3.4 Phase 2: Training and Validation 29
5.3.5 Phase 3: Testing and Evaluation 29

6 Results **30**

7 Discussion **39**
7.1 Limitations 41

8 Conclusion and Future Work **41**

1 Introduction

The stock market, a pivotal component of the global financial system, serves as a marketplace where investors can buy and sell shares of publicly traded companies [1]. Stocks, or equities, represent ownership in a company and entitle the holder to a proportion of the company's profits, either through dividends—payments made by a corporation to its shareholders—or through capital appreciation, which is the increase in the value of a stock [2]. Stock markets offer liquidity and establish a market price for a company's shares. An investment is considered liquid if it can be sold quickly and easily at a price close to the current market price [3].

The complexity of the stock market is influenced by many factors including economic indicators, investor behavior and geopolitical events [4]. Economic indicators such as inflation rates, employment figures, and GDP growth provide insights into the health of an economy and can significantly impact stock prices [5]. Similarly, investor behavior, driven by psychology and market sentiment, often leads to market trends and price movements that can be difficult to predict [2]. Geopolitical events, ranging from elections to international conflicts, also play a crucial role in shaping market dynamics [5].

The financial market is very broad and covers a variety of instruments such as bonds, commodities and derivatives but the stock market often captures the most attention due to its potential for high returns [2]. Investing in stocks can offer significant financial rewards, which is why it attracts a diverse range of participants, from individual investors to large institutional players [6]. However, with the potential for high returns comes the risk of losses [7], making the predictability of stock returns a topic of interest and debate.

Many researchers and analysts believe that a security's past price behavior contains valuable information about its future performance [8]. They argue that history tends to repeat itself, with patterns in past price movements likely to recur. "By carefully analyzing price charts and developing an understanding of these patterns, it is believed that one can predict the future behavior of prices and thereby increase expected gains"[8]. As a result, this area has attracted extensive research, with a strong focus on developing models and strategies to forecast market trends and enhance investment decisions.

In recent years, the convergence of machine learning and stock market analysis has given rise to a "sub-field dedicated to stock market prediction" [9]. Financial institutions are increasingly leveraging machine learning to make more informed trading decisions with greater ease. This technology provides investors with a competitive edge and enhances their understanding of market dynamics. Machine learning algorithms "can process vast amounts of stock data, identify complex patterns, and make accurate predictions" [10].

Given the complexity and dynamic nature of the stock market, the use of machine learning offers a way to improve predictions and investment strategies. This study will explore how machine learning models can help predict stock prices. By using advanced algorithms, this research aims to understand how machine learning can be used to forecast a stock's price. The following research questions will guide this investigation into the application

of machine learning in stock price prediction.

The main research question guiding this study is: Which machine learning model most effectively predicts stock prices?
To address this question, the study will focus on evaluating the effectiveness of different machine learning models, such as Long Short-Term Memory (LSTM) and Support Vector Machines (SVM), in predicting stock prices based solely on historical data.

Specifically, this research will address the following sub-questions:

Sub research question: How does the length of the data history impact the performance of each model?
This study aims to investigate how the length of data history impacts the performance of each model. Additionally, this study will test which model performs the best in both short-term and long-term forecasting scenarios. By analyzing different historical data lengths, the goal is to determine the optimal amount of historical data needed for accurate predictions.

Changing the historical data length is important for several reasons. Firstly, shorter historical data may capture recent market trends and patterns more effectively, which can be beneficial in volatile markets where recent information is more predictive. However, shorter datasets may also lead to overfitting [11], where the model performs well on the training data but poorly on unseen data due to a lack of diverse scenarios.

On the other hand, more comprehensive historical data can provide a more detailed view of market behavior. It allows analysts to detect seasonality, which refers to the periodic and predictable patterns that recur at fixed intervals within a time series. Similar to seasonality, cyclic behaviors exhibit repeated fluctuations in data over time and can reveal long-term trends that might be missed with shorter datasets [12].

This broader historical perspective can improve the model's ability to generalize to new data by exposing it to a wider variety of market conditions. However, including too much historical data can introduce outdated information, potentially reducing the model's relevance to current market dynamics.

Additionally, different financial models may have varying requirements for historical data length to perform optimally. For example, machine learning models like LSTM networks, which are designed to capture temporal dependencies, might benefit from longer sequences of data to effectively learn long-term dependencies. Conversely, simpler models like SVM might not require extensive historical data to make accurate predictions [13].

Therefore, it's important to try using different amounts of historical data to find the right balance between getting useful market information and not adding too much noise or overfitting the model. This exploration helps in understanding the trade-offs involved and in identifying the optimal data length that maximizes predictive accuracy for different models.

Sub research question: Which optimal parameters give the best performance of each predictive model?
This question seeks to identify the specific parameters and their values that enhance the performance of different predictive models. In machine learning, models often have several parameters, known as hyperparameters, which need to be tuned to achieve the best results. The goal is to determine which combinations of these parameters result in the most accurate and reliable predictions. By finding the optimal settings for each model, their efficiency and effectiveness can be improved in predicting stock prices.

This research will provide a comparative analysis of two models, examining their performance with short-term and long-term data lengths and discuss which model performs the best under different settings. The insights gained aim to contribute to the development of more accurate and efficient predictive models, thereby aiding investors in making better-informed decisions.

2 Literature Review

Various studies have employed machine learning models to enhance the accuracy of stock price forecasts. Some research focused on single models, while others utilized hybrid approaches to improve prediction performance. This section reviews studies in stock price prediction using machine learning, particularly focusing on Long Short-Term Memory (LSTM) networks and Support Vector Machines (SVM) and other relevant approaches.

2.1 Single Model Approaches

Several studies have explored the use of individual machine learning models for stock price prediction. For instance, Fischer and Krauss [14] employed LSTM networks to predict stock prices and found that LSTM outperformed traditional models due to its ability to capture temporal dependencies in the data . Their study demonstrated that LSTM networks could effectively handle the sequential nature of stock price data, achieving a significant improvement in prediction accuracy. Another study by Moghar and Hamiche [15], focused on predicting stock market prices using LSTM Recurrent Neural Networks. They demonstrated that LSTM models effectively capture temporal dependencies in stock price data. They also highlighted the impact of differing data lengths along with training parameters, showing that appropriate time length and tuning can significantly enhance model performance. Moreover, Zhang et al. [16] explored the use of LSTM model for stock price prediction. The key findings of the study include the effectiveness of Stacked-LSTM models in capturing complex temporal dependencies in stock data, leading to improved prediction accuracy. The research demonstrated that the multi-layer structure of Stacked-LSTM networks enhances the model's ability to learn from historical stock prices and other financial indicators.

On the other hand, Kim [17] investigated the use of Support Vector Machines (SVM) and ANN's for financial time series forecasting. The study highlighted that SVMs outper-

formed the ANNs model in predicting the future direction of a stock market. The study reported that the best prediction performance obtained with SVM is 57.8% significantly above the 50% threshold and concluded that SVM's strong theoretical foundations in statistical learning, could provide robust predictions compared to traditional linear models. Similarly, Yang [18] conducted an empirical analysis of stock market prediction using SVM, showing that SVM models could achieve a prediction accuracy as high as 90%, further demonstrating the efficacy of SVM in financial forecasting. Moreover Mintarya et al. [19] conducted a systematic literature review regarding Machine learning approaches in stock market prediction. Research highlighted that LSTM and SVM are among the most used methods in stock price forecasting. A study by Cao and Tay [20] focused on predicting stock market prices using Support Vector Machine (SVM). The research aimed to leverage SVM'S capabilities in handling non-linear data for accurate stock price predictions. The authors compared their model with traditional regression models and demonstrated that the model provided superior predictive performance. By using historical stock prices and various technical indicators, the study highlighted the model's effectiveness in capturing complex market dynamics, thereby underscoring its potential as a powerful tool for financial market predictions.

2.2 Hybrid Model Approaches

Other researchers have combined multiple techniques to enhance prediction accuracy. Patel et al. [21] developed a fusion model combining SVM with other machine learning techniques, such as Random Forest and Artificial Neural Networks (ANN), to predict stock prices. Their study concluded that the hybrid models outperformed individual models, suggesting that integrating multiple approaches could capture different aspects of the data, thereby improving predictive performance. Sharma et al. [22] reviews current research on hybrid models, including GAN-based, LSTM-based, and neural network-based models, Soft Computing based, GRU based models to provide optimal results. They mainly use ensembled LSTM to predict the KSE-100 index of the Pakistan Stock Exchange and they propose a hybrid model based on CNN and a Bidirectional Gated Recurrent unit, with the Feature selection applied. They compared the proposed model against preexisting popular techniques such as SVM, Random Forest, and Decision Tree. The proposed method is found to be more efficient relative to other techniques. Moreover, Long et al. [23] developed a hybrid model called MVL-SVM for stock price trend prediction by integrating multi-view learning with a Support Vector Machine (SVM).

2.3 Comparative Studies

Several comparative studies have evaluated the performance of different machine learning models for stock price prediction. Kalyan et al. [24] investigated the comparative effectiveness of three predictive models – ARIMA, LSTM, and Random Forest – in forecasting stock prices of 'Tata Motors' and 'Infosys'. These models are trained on a diverse set of features including open, close, high, and low prices to capture the underlying market dynamics. The evaluation of model performance is centred on their ability to forecast stock

prices over varying prediction horizons. In another study, Huang et al. [25] compared various machine learning algorithms, including SVM, ANN, and k-Nearest Neighbors (k-NN), for stock price prediction. Their results showed that SVM achieved the highest prediction accuracy among the models tested. Bhattacharjee and Bhattacharja [26] findings include finding the best possible method to predict the closing prices of stocks through a comparative study between different traditional statistical approaches including Weighted Moving Average, Naive Approach and machine learning techniques including Linear Regression, K-Nearest Neighbors, Support Vector Machine, Random Forest, Long Short Term Memory are performed. Moreover, this has been done in terms of prediction performances. After studying all the methods individually, the machine learning approach, especially the neural network models are found to be the most accurate for stock price prediction. Jabed, M. I. K. [27] investigated three distinct models: Facebook Prophet, Long Short-Term Memory (LSTM) and the Random Forest Regressor model from Ensemble Learning. This study distinguishes between model performances using standard assessment metrics, using historical data from ten domestic and six worldwide companies.

3 Machine Learning Models

3.1 Overview of Recurrent Neural Networks (RNN's)

One of the most popular neural networks for sequences is the recurrent neural network (RNN). RNNs are a class of artificial neural networks where connections between nodes can create cycles, allowing the network to maintain a form of memory. This cyclic nature makes RNNs particularly effective for tasks that involve sequential data, such as time series prediction, natural language processing (NLP), and speech recognition.

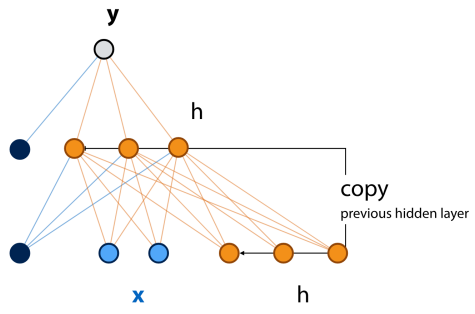

Figure 1: Configuration, taken from [28]

This diagram illustrates a basic recurrent neural network (RNN) architecture. In this configuration, the input vector x is extended by three nodes that connect to the hidden layer, h. The hidden layer, in turn, copies its state to the next time step, allowing the network to maintain a form of memory across sequences. This cyclic connection enables the RNN to process sequential data by taking into account the context provided by previous inputs.

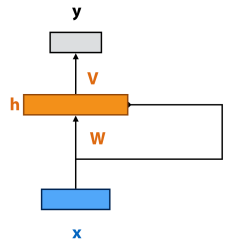

Figure 2: Visual shorthand, taken from [28]

This diagram represents a simplified representation of recurrent neural network (RNN) operations. The input vector x is processed and concatenated with the previous hidden state h. This combined input is then transformed using a weight matrix W. The resulting vector is passed through the hidden layer, which produces the next hidden state h and the output y.

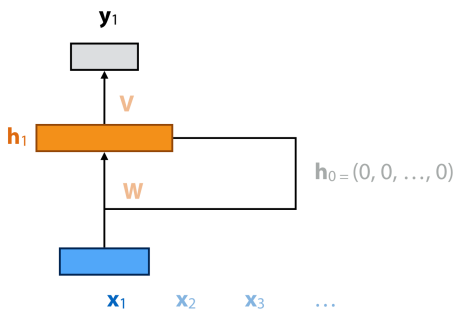

Figure 3: RNN's on sequences, taken from [28]

This diagram illustrates the application of a recurrent neural network (RNN) to a sequence of inputs. The initial hidden state $h0$ is typically initialized to a vector of zeros. The first input, $x1$, is fed into the network, where it is processed together with $h0$, using the weight matrix W. This results in the first hidden state $h1$, which captures the information from $x1$. The hidden state $h1$ is then used to produce the first output $y1$ through the transformation defined by the weight matrix V.

For subsequent inputs, the process is repeated in a similar manner. The next input $x2$ is fed into the network along with the hidden state $h1$. The RNN processes $x2$ and $h1$ using the weight matrix W to produce the next hidden state $h2$. This hidden state $h2$ is then used to generate the output $y2$. The sequence continues with input $x3$, which, together with $h2$, produces $h3$ and output $y3$, and so on.

This process demonstrates how RNNs handle sequential data by maintaining a hidden state that evolves with each new input, allowing the network to learn temporal dependencies and patterns within the sequence.

Training of Recurrent Neural Networks (RNN's)

The training process for recurrent neural networks (RNNs) starts with supplying the network with an input sequence x and a corresponding target sequence t. The input sequence represents the data that the RNN will process, such as series of stock prices over time. The target sequence, on the other hand, consists of the desired outputs that the network should learn to predict. This can be the stock price for the next day.

Once the input and target sequences are provided, the core algorithm used to train RNNs is Backpropagation Through Time (BPTT). BPTT works by unrolling the RNN through time, effectively transforming it into a deep feedforward neural network where each layer corresponds to a time step of the input sequence.

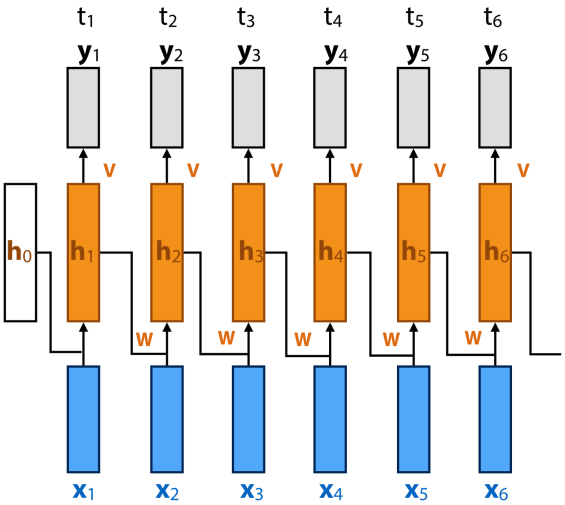

Figure 4: Unrolling, taken from [28]

Unrolling an RNN involves expanding the recurrent connections into a series of feedforward connections. Instead of visualizing a single small network applied repeatedly at every time step, the RNN is unrolled into multiple layers, with each layer corresponding to a time step. In the unrolled network, each step in the sequence is processed in parallel, with each step being a copy of the network. This unrolled form makes it easier to apply learning algorithms and visualize the flow of information. Each time step's network receives input from the previous time step's hidden state, maintaining a form of memory across the sequence.

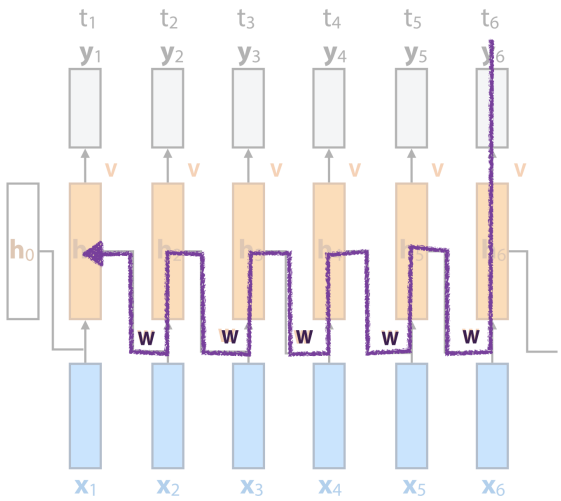

Figure 5: Backpropagation through time (BPTT), taken from [28]

During training, the error gradients are backpropagated through time. This process involves propagating the gradients from the output layer back through each time step in the sequence, updating the weights based on the error at each step. The gradients are calculated by comparing the predicted outputs to the target sequence and adjusting the weights to minimize the overall error. This method allows the network to learn from both short-term and long-term dependencies in the data.

Vanishing Gradient Problem

One of the significant challenges with BPTT is the vanishing gradient problem. As the gradients are propagated backward through time, they can become very small, diminishing as they reach earlier layers (time steps). This reduction in gradient size makes it difficult for the RNN to learn long-term dependencies, as the influence of earlier inputs on the current prediction becomes negligible. The vanishing gradient problem makes it difficult for the network to retain information over long sequences.

Selective Memory

Due to the vanishing gradient problem, basic RNNs can struggle to remember information over long sequences. While theoretically, RNNs can learn long-term dependencies, in practice, they need to be selective about what information to remember. Maintaining long-term memory for all information is not feasible, so the network must learn to focus on key pieces of information that are critical for the task at hand. This selectivity ensures that the network can retain essential information while discarding less relevant details.

3.2 Overview of Long Short-Term Memory (LSTM)

One significant challenge with traditional neural networks, especially recurrent neural networks (RNNs) as its mentioned before, is their difficulty in remembering information over long sequences. As a result, Long Short-Term Memory (LSTM) networks were introduced by Hochreiter and Schmidhuber as a solution to the vanishing gradient problem. LSTMs solve this problem through their unique architecture, which includes a cell state and three types of gates: input, forget, and output gates. All gates independently process the same initial inputs and information is propagated through the cells over time, and each cell produces a hidden state based on the current input and the hidden state from the previous cell. Then these inputs are separately processed through their respective weight matrices and activation functions.

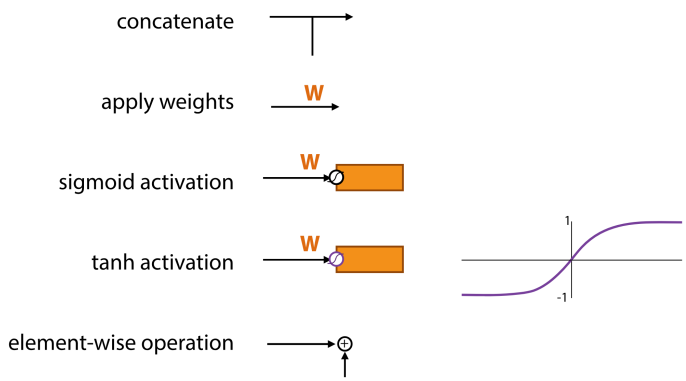

Figure 6: Notations used in the architecture of LSTM, taken from [28]

- *Concatenate:* This operation combines two or more vectors into a single vector. In the context of LSTMs, concatenation typically merges the input vector with the previous hidden state.
- *Apply Weights:* This represents the application of a weight matrix to the input vector. The weight matrix (W) is used to transform the input through a linear transformation, which is essential for adjusting the input dimensions and integrating the learned parameters.
- *Sigmoid Activation:* The sigmoid activation function squashes the input values to a range between 0 and 1. In LSTMs, the sigmoid function is used in the gates (input gate, forget gate, and output gate) to control the flow of information. The sigmoid output determines how much of each component should be passed through the gate.
- *Tanh Activation:* The tanh activation function squashes the input values to a range between -1 and 1. In LSTMs, the tanh function is used to create the new candidate values that could be added to the cell state. It helps in normalizing the output and maintaining the stability of the network.
- *Element-wise Operation:* This operation performs element-wise addition, multiplication, or other mathematical operations. In LSTMs, element-wise operations are

used to update the cell state by combining previous cell states with new candidate values scaled by the gates.

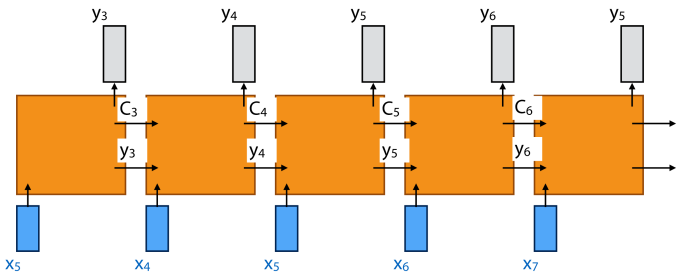

Figure 7: Sequence of cells, taken from [28]

This diagram represents a sequence of cells, part of a Long Short-Term Memory (LSTM) network. Each orange block symbolizes a cell in the network, processing inputs and maintaining a state over time. The inputs to the cells are denoted as $X5$, $X4$, $X6$, $X7$ and the cell outputs are labelled as $y3$, $y4$, $y5$ and $y6$.

The arrows show the flow of information: inputs (blue) enter the cells and the outputs (gray) are produced from each cell. The cell states, labeled as $C3$, $C4$, $C5$ and $C6$, propagate through the network, carrying information from one cell to the next one. This propagation of cell states is important for maintaining context and enabling the network to learn temporal dependencies.

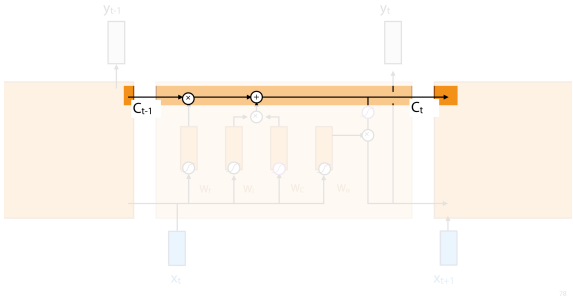

Figure 8: Usage of Conveyer Belt, taken from [28]

The diagram features the concept of the "conveyor belt" within an LSTM cell. This conveyor belt represents the cell state, which passes the previous cell state *Ct-1* to the next cell Ct. As the state flows through, the current input can modify it, allowing for the integration of new information. A notable aspect of this conveyor belt is the lack of activation functions along the direct path from previous cell state to the next cell. This absence means that gradients do not decay along this route, which helps in preserving long-term dependencies during backpropagation through time.

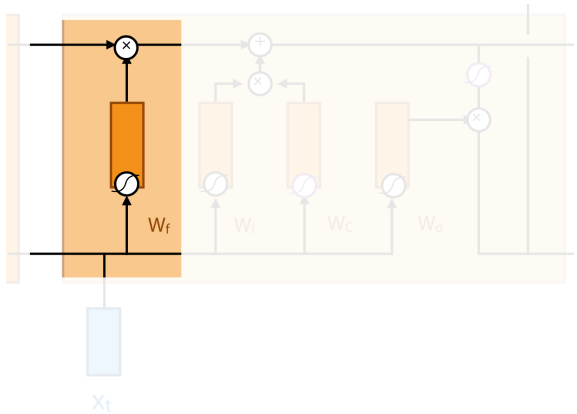

Figure 9: Forget gate mechanism, taken from [28]

The forget gate is the first modification done to conveyor belt. It determines how much of the previous cell state should be retained or discarded before passing it on to the next cell. The forget gate takes previous output hidden state and the current input, it concatenates these two vectors. This concatenated vector is then processed through a weight matrix. Then the sigmoid function outputs values between 0 and 1, representing the "forget" decision. A value close to 0 means "completely forget" the corresponding part of the cell state, while a value close to 1 means "completely retain" it.

Finally the resulting forget gate output is then multiplied element-wise with the previous cell state. This operation arranges the cell state by retaining important information and discarding irrelevant information.

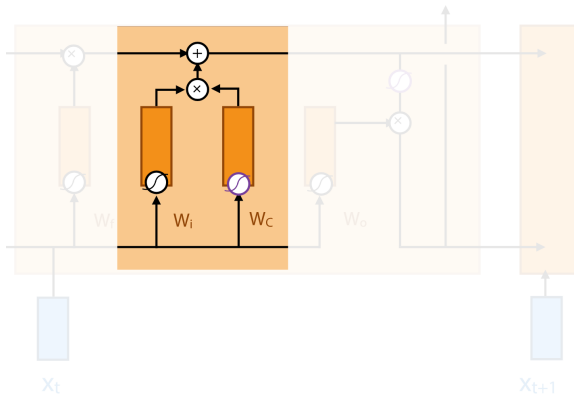

Figure 10: Input gate mechanism, taken from [28]

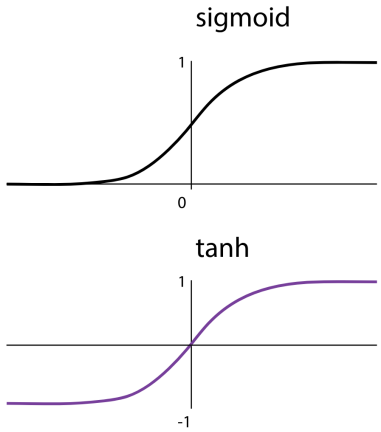

Figure 11: Sigmoid and Tanh activation functions, taken from [28]

Similar to forget gate, the input gate takes the previous hidden state and the current input. These inputs are concatenated and fed into two separate transformations. The concatenated inputs are passed through a weight matrix Wi and then through a *sigmoid* activation function. This produces an output with values between 0 and 1. This vector determines how much of the new information should be added to the cell state. Simultaneously, the concatenated inputs are passed through another weight matrix Wc and then through a *tanh* activation function. This generates the candidate cell state with values between -1 and 1, representing the new information to be potentially added to the cell state.

The output of the *sigmoid* function (left activation) is element-wise multiplied with the candidate cell state (right activation). This multiplication controls how much of the candidate state should be added to the cell state, effectively arranging the new information's impact based on its importance.

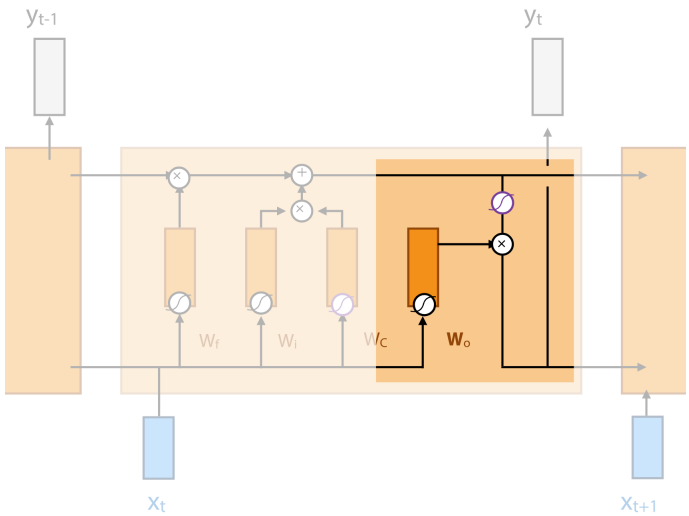

Figure 12: Output gate mechanism, taken from [28]

The output gate in an LSTM cell decides what information to output and pass to the next time step. Similar to the forget and input gates, the output gate receives the previous hidden state and the current input. These inputs are concatenated and processed through a weight matrix Wo. This is followed by a sigmoid activation function then the current cell state is passed through a tanh activation function. The tanh-activated cell state is then element-wise multiplied by the sigmoid output, filtering and scaling the cell state values based on their importance. The resulting vector from this multiplication is sent out as the current output yt. Which serves both as the LSTM's prediction for the current time step and as the input to the next LSTM cell in the sequence. This gating mechanism ensures that only relevant information is propagated forward, enabling the LSTM to maintain long-term dependencies effectively.

In LSTM networks, the gradients that pass through the cell state, referred as the conveyor belt, tend to preserve long-term dependencies because this path involves fewer transformations and primarily element-wise operations, which helps prevent the vanishing gradient problem. In contrast, the gradients passing through the hidden states are more likely to diminish over time due to multiple transformations and non-linear activations such as sigmoid and tanh, which can lead to the vanishing gradient problem. This distinction allows the cell state to effectively carry long-term information, while the hidden state is better suited for capturing short-term dependencies.

3.3 Overview of Support Vector Machine (SVM)

Support Vector Machines (SVMs) are a powerful set of supervised learning algorithms used for classification and regression tasks. They are particularly effective in high-dimensional spaces and are known for their effectiveness in handling both linear and non-linear classification problems.

Fundamental Concepts

The primary objective of SVMs is to identify the optimal hyperplane that best separates data points belonging to different classes.

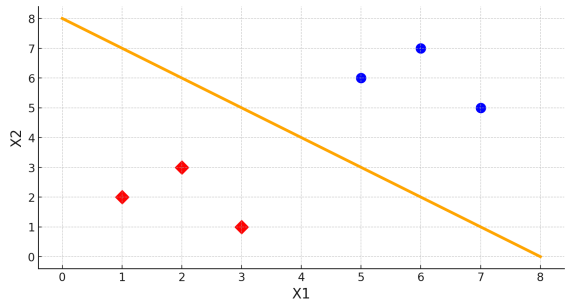

Figure 13: Hyperplane

In a two-dimensional space, a hyperplane is represented by a line that separates the data points into distinct classes. Similarly, in N-dimensional space, a hyperplane has (N-1)-dimensions [29]. Data points located on one side of the hyperplane are assigned to one class, whereas those on the opposite side are assigned to a different class [29].

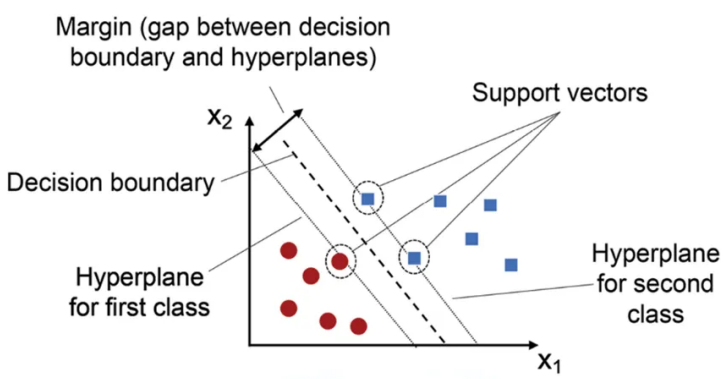

Figure 14: Maximum Margin Hyperplane, taken from [29]

The margin in SVMs refers to the distance between the hyperplane (the decision boundary) and the nearest data points from each class, known as support vectors. The SVM algorithm aims to find the hyperplane that maximizes this margin. A larger margin is desirable because it increases the classifier's robustness and generalizability. This means that the classifier is less likely to misclassify new data points, as it ensures a greater separation between classes, reducing the risk of overlap or misclassification. By maximizing the margin, SVM seeks to find the most optimal hyperplane that not only separates the classes but also enhances the model's predictive performance on unseen data.

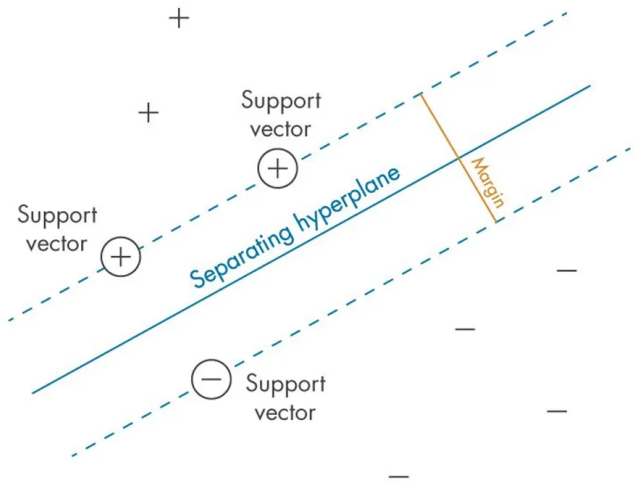

Figure 15: Support vectors, taken from [29]

Support vectors are the data points closest to the decision boundary in a SVM model. These points are critical because they define the position and orientation of the decision boundary. The support vectors are the only points that influence the decision boundary directly and any other points not on the margin do not affect the hyperplane.

The robustness of the SVM model is enhanced by this characteristic, as outliers that do not lie on the margin do not impact the decision boundary. Mathematically, in a linearly separable scenario, the decision boundary can be represented by the equation.

$$w \cdot x + b = 0 \tag{1}$$

where w is the weight vector, x is the feature vector and b is the bias term. The margins are given by

$$w \cdot x + b = \pm 1 \tag{2}$$

and the support vectors lie on these margins. The objective of the SVM is to maximize the distance

$$\frac{2}{\|w\|} \tag{3}$$

between the margins, subject to the constraint that all training data points are correctly classified.

The Kernel Trick

In cases where data is not linearly separable, SVM uses kernel functions to transform the input data into a higher-dimensional space, making it easier to separate classes. This capability allows SVM to effectively address complex non-linear problems. The role of support vectors remains unchanged, but the hyperplane is established in this higher-dimensional space, allowing SVM to find a hyperplane that separates the data with the greatest margin. The kernel trick enables SVMs to operate in high-dimensional spaces without explicitly mapping data points to those spaces, by computing the necessary dot products in the original space, making the computations feasible and efficient.

As mentioned, SVMs transform the data into a higher-dimensional space where it becomes linearly separable. This transformation is done using a mapping function

$$\phi(x)$$

which maps the original features x to a higher-dimensional space. To find the optimal hyperplane, SVMs need to compute the dot product between pairs of data points after they have been transformed. If x_i and x_j are two data points in the original space, the dot product in the transformed space is

$$\phi(x_i) \cdot \phi(x_j)$$

Directly computing $\phi(x)$ for every data point and then finding the dot product in this high-dimensional space can be computationally expensive and sometimes infeasible, especially when the dimension is very high or infinite.

This is where the kernel trick comes in. A kernel function

$$K(x_i, x_j)$$

computes the dot product $\phi(x_i) \cdot \phi(x_j)$ directly in the original space without ever explicitly computing $\phi(x)$. The kernel function effectively acts as a shortcut.

Common Kernel Functions

1. Linear Kernel: $K(x_i, x_j) = x_i \cdot x_j$. This is a simple dot product between two input vectors and is used when the data is already linearly separable.

2. Polynomial Kernel: $K(x_i, x_j) = (x_i \cdot x_j + c)^d$, where c and d are constants. This kernel allows the algorithm to fit polynomial boundaries.

3. The Radial Basis Function (RBF) or Gaussian kernel: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, where γ is a parameter. This kernel maps the data into an infinite-dimensional space and is very effective for non-linear problems.

4. Sigmoid Kernel: $K(x_i, x_j) = \tanh(\alpha x_i \cdot x_j + c)$, where α and c are parameters. This kernel is similar to the neural network activation function.

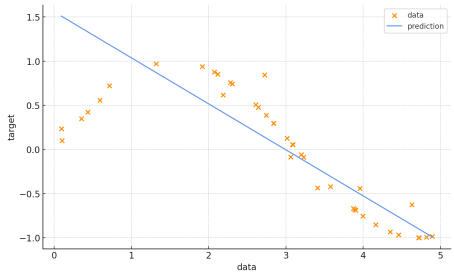

Figure 16: Linear Kernel, adapted from[1]

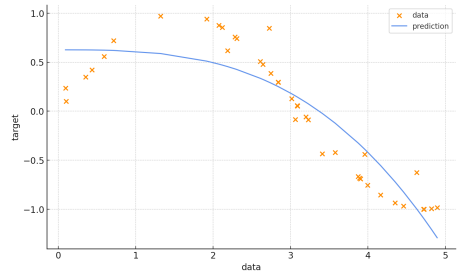

Figure 17: Polynomial Kernel, adapted from[1]

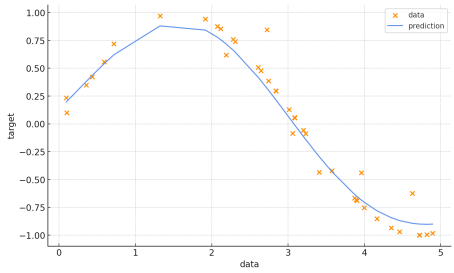

Figure 18: RBF/Gaussian Kernel, adapted from[1]

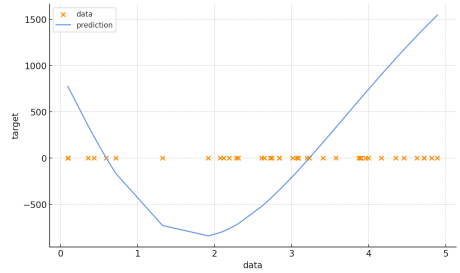

Figure 19: Sigmoid Kernel, adapted from[1]

Mathematical Perspective

If x_i and x_j are data points, and ϕ is the mapping function to a higher-dimensional space, the kernel function K satisfies:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

By using K instead of explicitly computing $\phi(x_i)$ and $\phi(x_j)$, Support Vector Machines (SVMs) can work with complex transformations without the associated computational overhead.

3.4 Support Vector Regression (SVR)

Support Vector Regression (SVR) is a machine learning method tailored for regression problems, where the goal is to predict continuous numeric values rather than categorical outcomes. As a variant of Support Vector Machines (SVM), SVR is particularly effective in handling complex relationships between variables and is widely used in various applications such as time series forecasting, stock price prediction and environmental modeling.

SVR works by fitting the best possible line (or hyperplane in higher dimensions) that approximates the relationship between input features and the target variable, within a specified margin of tolerance, denoted as epsilon, ϵ. The key idea is to ensure that most

[1] https://www.geeksforgeeks.org/support-vector-regression-svr-using-linear-and-non-linear-kernels-in-scikit-learn/

data points lie within this margin, effectively minimizing the prediction error while avoiding overfitting.

- Margin in SVR: In support vector regression, a margin also referred to as a tube is established around the regression hyperplane. The hyperplane is the model's best fit line or curve that predicts the target variable from the input features.
  This margin is essentially a region around the hyperplane where errors are tolerated without penalty. It is defined by the epsilon ϵ parameter.

- Epsilon Parameter (ϵ): The epsilon parameter specifies the width of the tube or margin and it creates a band around the regression hyperplane within which deviations of the predicted values from the actual values are not penalized.
  If the predicted value for a data point lies within this epsilon-tube, the prediction is considered acceptable, and no penalty is imposed.

  By focusing only on errors that exceed the epsilon margin, the model becomes more robust. It ignores minor discrepancies that are likely due to noise in the data and only concentrates on significant deviations. This can prevent the model from overfitting to minor fluctuations and noise in the training data.

Support Vector Regression (SVR) Basics

Support Vector Regression (SVR) aims to find a function that predicts target values as accurately as possible while allowing for some errors within a specified range.

$$\min \frac{1}{2}\|w\|^2$$

This objective function is trying to make the weight vector w as small as possible. Smaller weights mean a simpler model that is less likely to overfit the data.

$$|y_i - (w \cdot x_i + b)| \leq \epsilon$$

This constraint ensures that the predicted value $w \cdot x_i + b$ (where $w \cdot x_i$ is the dot product of w and x_i, and b is a bias term) is within ϵ of the actual value y_i.
This means that as long as the model's predictions are within an acceptable range (the epsilon-tube) from the actual values, they are considered correct. No penalty is given for these small errors.

It's often impossible to fit all data points within the epsilon-tube because of noise and outliers. This is where slack variables come in.

$$\min \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*)$$

This objective function now includes ξ_i and ξ_i^*, which are slack variables that allow for some errors to be outside the epsilon-tube. The term $C\sum_{i=1}^{n}(\xi_i + \xi_i^*)$ adds a penalty for

these errors.

By allowing some points to be outside the epsilon-tube and penalizing them, the model can still find a good balance between fitting the data and maintaining simplicity. The parameter C controls how much penalty is given to these errors, balancing between fitting the training data well and keeping the model simple.

$$\begin{aligned} y_i - (w \cdot x_i + b) &\leq \epsilon + \xi_i \\ (w \cdot x_i + b) - y_i &\leq \epsilon + \xi_i^* \\ \xi_i, \xi_i^* &\geq 0 \end{aligned}$$

These constraints ensure that the predictions can be outside the epsilon-tube by an amount determined by the slack variables ξ_i and ξ_i^*. This flexibility allows the model to handle data points that are noisy or outliers without being forced to fit them perfectly, which could lead to overfitting.

4 Dataset

To analyze the stock performance of a major technology company, historical stock data for Apple Inc. was retrieved using the open-source Python library 'yfinance' from the Yahoo Finance website. The first dataset spans from 2014-01-01 to 2024-01-01 (10 years) and the second one spans from 2019-01-01 to 2024-01-01 (5 years). It includes the following attributes for the stock: High, Low, Open, Close, Adjusted Close, and Volume, the details can be found in the Section 5.2.

The decision to focus exclusively on the stock of a single company, specifically Apple Inc., was made to minimize the confounding influence of market volatility and external macroeconomic factors, thereby enhancing the clarity and precision of the analysis. By concentrating on a single stock, this study aims to isolate the effects of varying historical data lengths on the model's predictive performance, ensuring that the observed outcomes are more directly attributable to the variables under investigation. Apple Inc.'s stock was selected due to its substantial historical data and unique overall upward trend, providing a robust and informative case study. Although Apple's stock price development may exhibit atypical characteristics, this presents a valuable opportunity to challenge the models' abilities and assess their performance under unexpected and complex patterns. This testing under unique conditions allows for a more nuanced understanding of how different historical data lengths impact the models' effectiveness.

Additionally, by focusing on a single stock, the analysis avoids the complexities introduced by sector-specific factors. Sector-specific factors refer to the unique characteristics and events that can affect stocks within a particular industry. Different sectors can respond differently to macroeconomic conditions, policy changes, or industry-specific news. For instance, the technology sector might be more sensitive to changes in innovation and regulatory policies, while the healthcare sector could be more affected by changes in healthcare legislation and pharmaceutical breakthroughs. These sector-specific influences add another layer of complexity to the analysis, as each sector may require different modeling

approaches to capture these unique dynamics accurately.

	Open	High	Low	Close	Adj Close	Volume
Date						
2024-01-02	187.149994	188.440002	183.889999	185.639999	185.403412	82488700

Figure 20: Raw data of Apple Inc. stock

5 Methodology

5.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is performed to visualize and understand the features of the dataset. Key visualizations include the "Close Price Over Time" plot, which highlights trends and fluctuations in the stock price over the dataset period. Figures 22 and 23 specifically show the price movements of Apple (AAPL) over 10 years and 5 years, respectively, allowing for a comparative analysis of long-term and short-term trends. The "Volume Over Time" plot provides insights into trading activity and liquidity, while the "High and Low Prices Over Time" plot captures the price range and volatility, illustrating the extremes of market behavior. Additionally, the "Open Price Over Time" plot indicates the initial market sentiment for each trading day. These visualizations help identify patterns, trends, and potential anomalies in the data, offering a comprehensive understanding of the stock's historical performance.

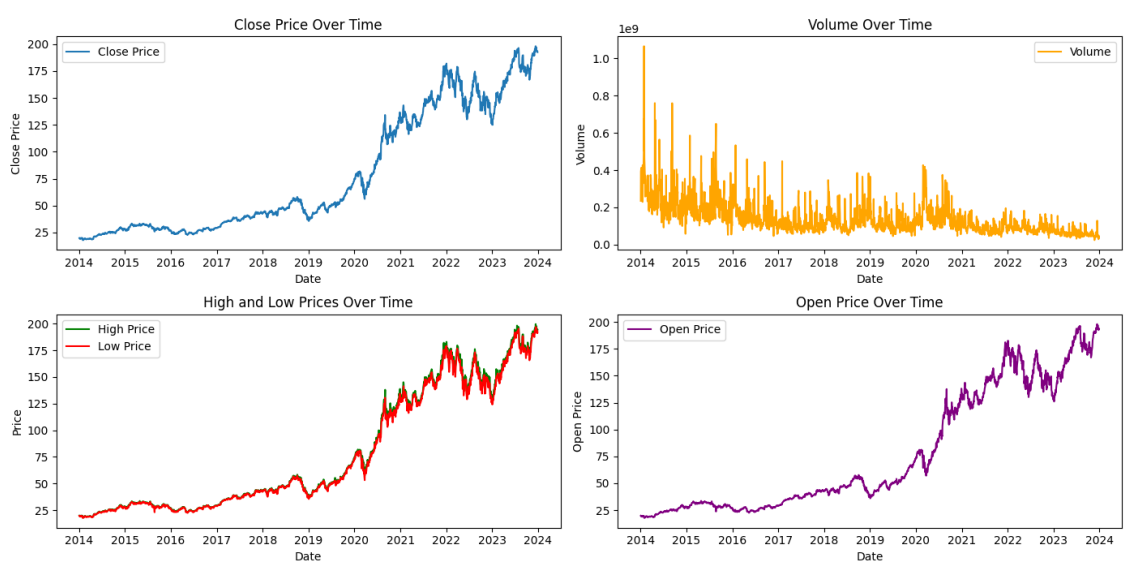

Figure 21: Visualizations of stock data over time, including the Close Price, Volume, High and Low Prices, and Open Price.

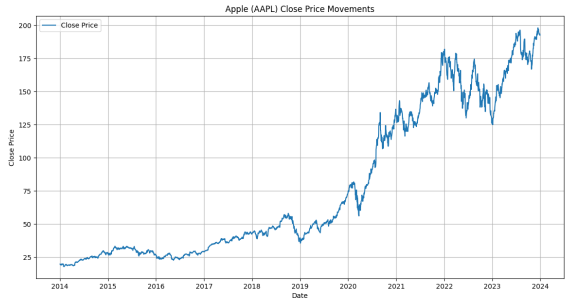

Figure 22: The plot shows the price movements of Apple (AAPL) over 10 years

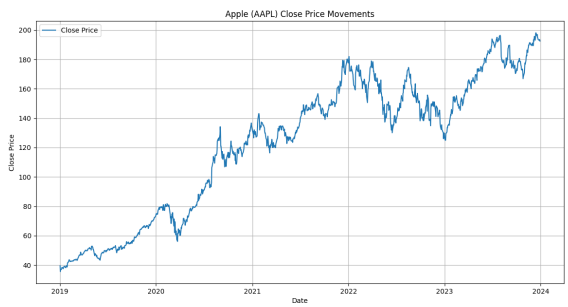

Figure 23: The plot shows the price movements of Apple (AAPL) over 5 years

5.2 Features

All of these features were selected based on their potential influence on the stock's closing price, which is the target variable. Based on numerous studies that have successfully utilized Open, High, Low, Adjusted Close, Close, Volume [30, 31, 32, 33, 34, 35] data in their predictive models, these features have been selected for use in this study.

- Close: The closing price of the stock on each trading day. Close price is directly related to the target variable.
- Adjusted close: Adjusted close is the closing price after adjustments for all applicable splits and dividend distributions. Closing price simply refers to the cost of shares at the end of the day, the adjusted closing price takes dividends, stock splits into account therefore it makes it a valuable feature.
- Open: The opening price of the stock on each trading day.
- High: The highest price of the stock on each trading day.
- Low: The lowest price of the stock on each trading day. Open, High and Low prices provide insights into the stock's price movement within a trading day.

- Volume: The number of shares traded on each trading day. Volume indicates the trading activity and can reflect the stock's liquidity.

5.3 Experimental Setup

This experimental setup is done for each model with 5 years of data and 10 years of data. Initially all the relevant libraries are imported, for LSTM: yfinance [36], matplotlib [37], numpy [38], torch'[39]. For SVR: yfinance [36], matplotlib [37], sklearn [40], numpy [38].

5.3.1 Data Preprocessing

The raw stock data undergoes several preprocessing steps to ensure its suitability for model training and evaluation. Initially, the data is inspected for any missing values, and there weren't any missing values. The preprocessing pipeline involves generating sequences and corresponding targets from the raw data. Specifically, for each sequence of a given length, the subsequent data point is designated as the target, with the 'Close' price of the stock as the target variable.

Subsequently, the sequences and targets are converted into arrays for efficient computation. Feature scaling is applied to the sequences using standardization, which involves calculating the mean and standard deviation of the sequences and then normalizing them. This standardization process ensures that the data has a mean of zero and a standard deviation of one, facilitating better model performance. The targets are also normalized by calculating and applying their mean and standard deviation. The preprocessing function returns the standardized sequences and normalized targets along with their respective means and standard deviations, thus preparing the data for subsequent model training and evaluation steps.

For LSTM (Long Short-Term Memory) networks, it is essential to provide the input as sequences of time steps with multiple features. This model requires the input data to be in a specific shape: [samples, time steps, features]. Batch size represent the number of sequences processed in parallel, and time steps correspond to the length of each consecutive sequence, and features are the individual data points in each sequence (e.g., Open, High, Low, Close, Adjusted close, Volume). By organizing the data in this format, the LSTM can effectively learn the temporal dependencies and patterns within the data, leading to more accurate predictions. The preprocessed sequences, standardized and appropriately shaped, are then fed into the LSTM model for training and evaluation.

For the Support Vector Regression (SVR) model, the input data undergoes a specific transformation to meet the model's requirements. Initially, the sequences, which are in a three-dimensional format (samples, time steps, features), are reshaped into a two-dimensional format (samples, features * time steps). This process effectively flattens each sequence into a single row, concatenating all the time steps. The reshaping is performed using the reshape method, where the first dimension (number of samples) remains unchanged, and the second dimension is calculated as the product of the number of time

steps and features. This transformation ensures that the input data is suitable for the SVR model, which expects a two-dimensional array.

5.3.2 Data Splitting

After the preprocessing and normalization steps, the dataset is divided into three subsets: 70% for training, 20% for validation, and 10% for testing. Allocating a larger portion of the data to the training set is advantageous, as it allows the model to learn from a more extensive range of examples, thereby enhancing its learning capability. The validation set is used to adjust the model parameters and prevent overfitting by providing information on how well the model is performing during the training process. The testing set, comprising the smallest portion of the data, is used to evaluate the model's performance on unseen data. This assessment is important to adapt the generalization ability of the model. However, it is important to ensure that the testing set is not too small, as an inadequately sized test set may compromise the reliability of the model's performance evaluation and its predictive power.

5.3.3 Phase 1: Hyperparameter Tuning

The initial phase focused on creating new instances of the models using various hyperparameter combinations. Each combination was trained and validated to assess its performance. The performance was measured using the mean squared error (MSE) score. The hyperparameter combination that yielded the lowest MSE score was recorded as the best performer.

Initially, a comprehensive grid search is conducted over a broad spectrum of potential hyperparameter values, systematically exploring the parameter space to identify promising configurations. Grid search is an exhaustive search method that ensures all possible combinations within the specified ranges are evaluated. This thorough exploration yields a preliminary set of optimal parameters, providing a solid foundation for further refinement.

Grid search involves defining a discrete set of values for each hyperparameter and evaluating the model's performance for every possible combination of these values. Although this approach is computationally intensive, it guarantees that the best combination within the specified ranges is found. The main advantage of grid search is its thoroughness, but it can become impractical when dealing with a large number of hyperparameters or very fine-grained ranges due to the combinatorial explosion of possibilities.

Subsequently, the identified best parameters from the grid search are used to refine the hyperparameter ranges, narrowing them down for a more focused and efficient random search. Unlike grid search, random search does not evaluate all possible combinations but instead samples hyperparameter values from specified distributions within the refined ranges. This targeted sampling allows for a more efficient exploration of the parameter space, as it focuses on regions that are more likely to contain optimal values.

Random search's main advantage is its ability to find good hyperparameter values with less computational cost compared to grid search, especially when dealing with a large number of hyperparameters. By sampling randomly within the refined ranges, random search can often identify effective parameter values that might be missed by a grid search due to its fixed evaluation points.

This combined approach of initial grid search followed by random search leverages the strengths of both methods. The exhaustive nature of grid search ensures a thorough initial exploration, while the targeted and efficient nature of random search hones in on the most promising areas of the hyperparameter space. Ultimately, this strategy leads to the selection of the most effective parameter values, as measured by the lowest validation mean squared error (MSE).

Having established the methodology for hyperparameter tuning through a combined approach of grid search and random search, the subsequent sections will delve into the specific hyperparameters that were optimized for the LSTM and SVR models. This includes detailed descriptions of the chosen hyperparameters, their respective ranges, and the rationale behind their selection. By highlighting the tuned parameters, the study aims to provide a comprehensive understanding of the factors that contribute to the model's predictive performance. This detailed examination of the tuned parameters will offer valuable insights into the model's configuration, further elucidating the steps taken to achieve the best possible predictive accuracy.

Tuned Parameters, LSTM:

'hidden_size': The number of neuron units in the hidden layers of the neural network, which determines the network's capacity to learn and represent complex patterns. Each neuron processes a portion of the input sequence, contributing to the model's ability to capture and learn temporal dependencies.
range: [32, 64, 128, 256]
Given that the dataset contains six features, a relatively small feature set, smaller hidden layer sizes such as 32 and 64 were initially considered. Additionally, due to the limited number of samples in the dataset, larger hidden layer sizes beyond 256 were not explored.

'num_layers': This parameter refers to the depth of the network, with each layer potentially capturing different levels of temporal patterns in the data.
range: [1-4]
Due to the small sample size of the dataset, the range was kept small. Limiting the number of layers to a maximum of four ensured that the model complexity was kept manageable.

'dropout': A regularization technique used in training neural networks, including LSTM models, to prevent overfitting. It specifies the fraction of neurons to be randomly "dropped out" or ignored during each training iteration. By doing so, the model becomes less sensitive to the specific weights of individual neurons, thereby promoting generalization.
range: [0.1-0.5]

The standard range for dropout rates, typically between 0.1 and 0.5, was used to ensure effective regularization.

'learning_rate': The rate at which the model learns. It controls the step size at which the model's parameters are updated during the optimization process.
range: $[10^{-2} - 10^{-5}]$
The standard range for learning rates, was employed to ensure a balance between convergence speed and stability.

'sequence_length': Defines the length of input sequences for sequence models, impacting how much sequential data the model processes at once.
range: [10,20,30,40,50]
If the sequence length was set to 10, it would be too narrow to effectively capture the temporal dependencies in the data. Conversely, if it was set larger than 50, it would be too large, potentially introducing unnecessary complexity and computational overhead.

Fixed parameters:

'output_size': 1
The output_size is set to 1 because the model is designed to predict a single value for each input sequence. This is common in regression tasks where the goal is to forecast a specific numerical value.

'batch_size': 64
Batch size refers to the number of training examples utilized in one iteration of model training. A batch size of 64 strikes a good balance between training speed and model performance. Small batch sizes (e.g., 1 or 2) can lead to noisy gradients, where the directions for improving the model are erratic and inconsistent because the model updates based on too little data at a time. Conversely, large batch sizes (e.g., 1000) offer stability but are slow and require significant memory.

'num_epochs': 200
The num_epochs is set to 200 to ensure that the model has sufficient opportunities to learn from the data. An epoch is one complete pass through the entire training dataset. Training for 200 epochs allows the model to iteratively update its weights, improving performance over time until the model converges or no further significant improvements are observed.

Optimization: 'Adam':
The Adam optimizer is used for training because it is efficient and adapts the learning rate for each parameter automatically, making training faster and more effective. Adam combines the benefits of other optimizers, making it suitable for models like LSTMs that work with complex data and noisy gradients. According to Kingma et al. [41] this optimizer is "computationally efficient, has little memory requirement, invariant to diagonal rescaling of gradients and is well suited for problems that are large in terms of data/parameters". Additionally, Adam is less sensitive to hyperparameter settings, which simplifies the tun-

ing process and often leads to better performance.

The optimal configuration identified for 10 years data was a hidden size of 256, 2 LSTM layers, a dropout rate of 0.1, a learning rate of 0.001, and a sequence length of 30.

The optimal configuration identified for 5 years data was a hidden size of 256, 2 LSTM layers, a dropout rate of 0.1, a learning rate of 0.0005, and a sequence length of 30.

Tuned parameters, SVR:

Kernel: 'rbf': The Radial Basis Function (RBF) kernel is used, which is effective for non-linear data by mapping it into a higher-dimensional space.
The RBF kernel is known for its flexibility and ability to handle non-linear relationships in the data. Stock prices are influenced by numerous non-linear factors and the RBF kernel is well-suited to capture these complex, non-linear patterns, making it a preferred choice for stock price prediction. Empirical evidence supports the effectiveness of the RBF kernel in financial time series forecasting. Studies have shown that the RBF kernel often outperforms other kernels like linear and polynomial in capturing the intricacies of financial data due to its ability to model non-linear relationships efficiently [20, 17]. Additionally, compared to polynomial kernels, which require tuning additional parameters like the degree of the polynomial, the RBF kernel involves fewer hyperparameters (mainly C and gamma). This simplifies the hyperparameter tuning process and reduces computational overhead while still achieving high performance.

'C': The regularization parameter C controls the trade-off between achieving a low training error and a low testing error.
range: [1.0-100.0]
The min-max values of this range were kept as wide as possible to allow the model to explore a broad spectrum of regularization strengths.

'Gamma': The kernel coefficient for non-linear kernels. It defines how far the influence of a single training example reaches and determines the influence radius of each training point on the decision boundary.
range: [scale, auto]
These settings allow for flexibility in how the model learns the influence of each training example. The 'scale' setting uses $\frac{1}{n \times \text{var}}$ as the value of gamma, where n is the number of features and 'var' is the variance, ensuring an effective scaling based on the dataset. The 'auto' setting uses $\frac{1}{n}$, providing a straightforward approach. By exploring both options, the model can be tuned for optimal performance based on the characteristics of the dataset.

'Sequence_length': Sequence length refers to the number of previous time steps used as input features to predict future values.
range: [10,20,30,40,50]
For the SVR model, sequence length determines the number of past observations included in the input feature set. If the sequence length is too short (e.g. shorter than 10), it may not capture enough historical information to make accurate predictions. Conversely, if it

is too long (e.g., greater than 50), it can introduce unnecessary complexity and computational overhead.

The optimal configuration identified for 10 years data was a kernel: rbf, C: 15.0, gamma: auto and a sequence length of 10.

The optimal configuration identified for 5 years data was a kernel: rbf, C: 52.0, gamma: auto and a sequence length of 10.

5.3.4 Phase 2: Training and Validation

In the second phase, the models were initialized with the optimal hyperparameters identified in the first phase. These models were then trained using the training dataset and evaluated with the validation dataset. To enhance understanding, the training and validation processes were visually plotted, providing a clear depiction of the model's performance over time.

5.3.5 Phase 3: Testing and Evaluation

The final phase involved evaluating the models with the test dataset. The models, trained with the optimal parameters, were tested to determine their performance on unseen data. The test performance was also recorded to compare with the validation results to verify reliability. The results were to analyzed using the following metrics:

Mean Squared Error (MSE): This metric measures the average of the squares of the errors, which are the differences between the predicted and actual values. It emphasizes larger errors, making it useful for identifying models that produce large deviations.

Root Mean Squared Error (RMSE): This is the square root of the MSE, providing an error metric in the same unit as the target variable. It offers an interpretable measure of the average magnitude of the error.

Mean Absolute Error (MAE): This metric measures the average of the absolute errors, providing a straightforward interpretation of the average prediction error. It is less sensitive to outliers compared to MSE and RMSE.

6 Results

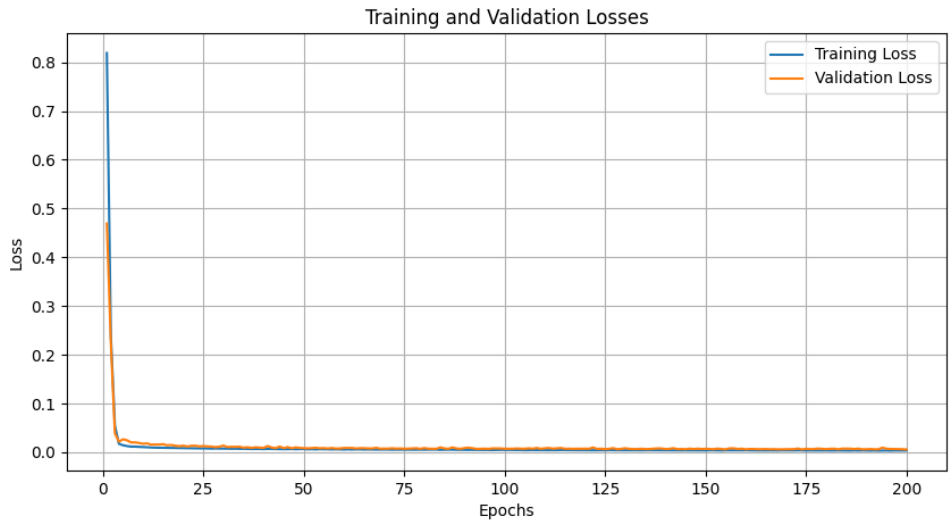

Figure 24: The learning curves obtained from 5 years of data of LSTM

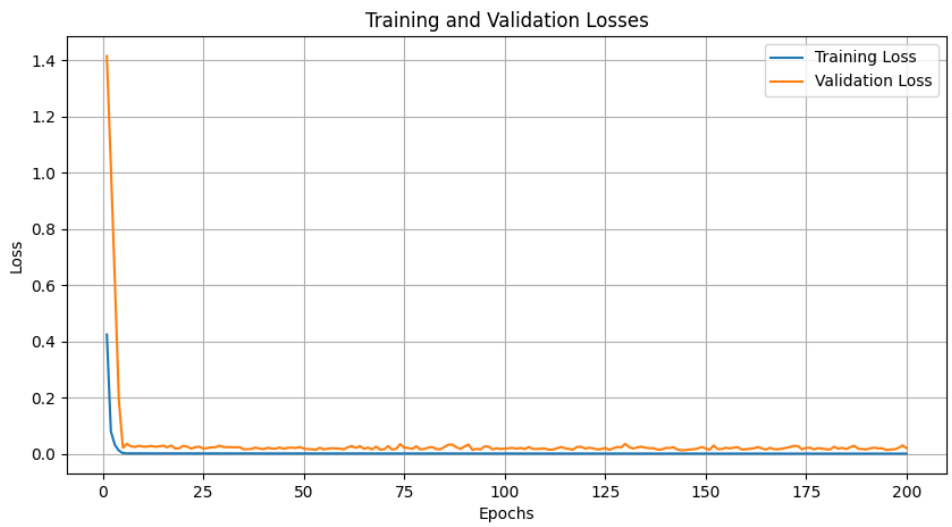

Figure 25: The learning curves obtained from 10 years of data of LSTM

After completing the training with optimized parameters for each dataset length, the learning curves are depicted in Figures 24 and 25.

For both datasets, the training process proceeded without significant issues, and the models demonstrated clear convergence. Notably, the models showed rapid convergence within the first 25 epochs. However, training was extended to 200 epochs to ensure the models achieved optimal performance and stability. As illustrated in the figures, the validation loss exhibits minimal fluctuations and remains consistent, eventually reaching a saturation point. This consistency in the validation loss, coupled with its alignment with the training loss, indicates that the models do not suffer from overfitting. Instead, the models maintain a robust generalization capability throughout the extended training period.

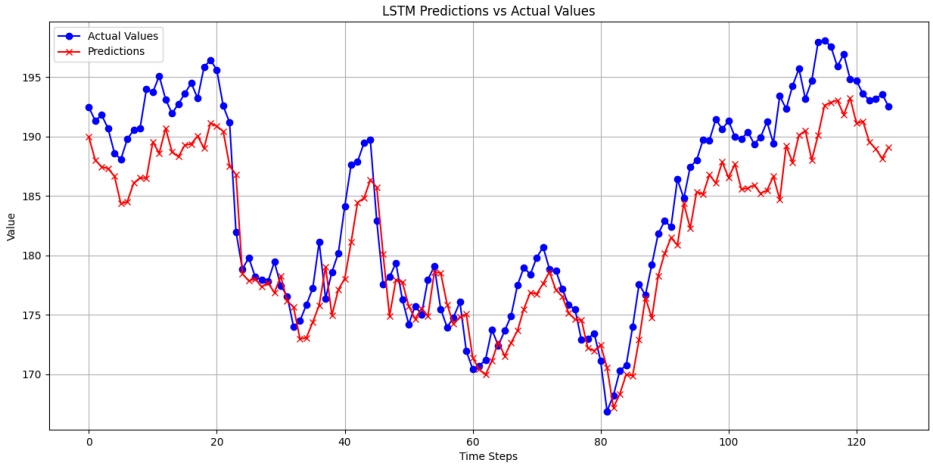

Figure 26: The plot displays the predictions versus the actual values for the LSTM model on the test data over a period of 5 years

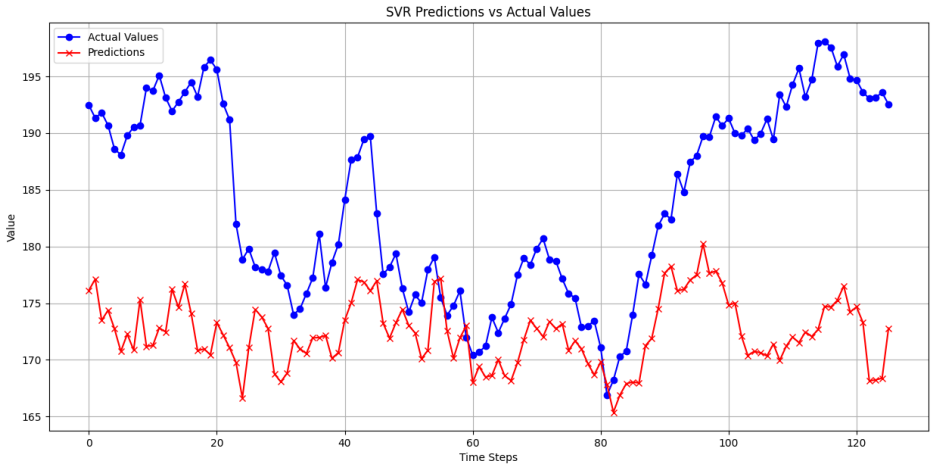

Figure 27: The plot displays the predictions versus the actual values for the SVR model on the test data over a period of 5 years

Prediction on the test data 5 years LSTM:

The Figure 26 represents time steps on x-axis, it can be interpreted as days and the actual closing price of the stock on y-axis. The LSTM predictions follow the actual values closely, capturing the overall trend well. There are some minor deviations, but the general pattern of peaks and troughs in the actual values is reflected in the predictions. The alignment between the blue points (actual values) and the red line (predictions) indicates a strong correlation, with both short-term fluctuations and long-term trends captured effectively.

Prediction on the test data 5 years SVR:

The Figure 27 shows the SVR's predictions with significant deviations from the actual values. The model consistently underestimates the actual values, demonstrating a lack of responsiveness to upward trends and fluctuations. During periods of sharp increases or decreases in the stock price, the SVR model's predictions do not follow the actual trend, leading to substantial errors. This underperformance suggests a bias in the model, preventing it from generalizing well to unseen data. The variability and systematic errors in the predictions highlight the SVR model's limitations in handling the complexities of stock price data over time.

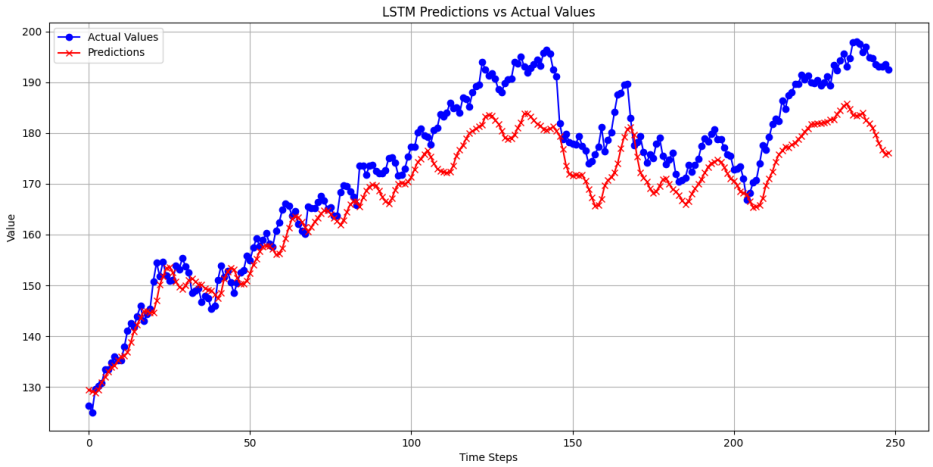

Figure 28: The plot displays the predictions versus the actual values for the LSTM model on the test data over a period of 10 years

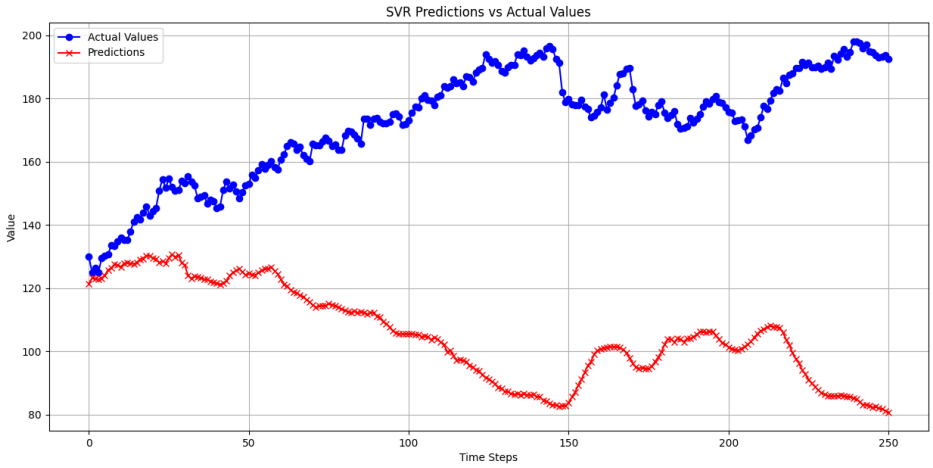

Figure 29: The plot displays the predictions versus the actual values for the SVR model on the test data over a period of 10 years

Prediction on the test data 10 years LSTM:

The Figure 28 represents time steps on x-axis, it can be interpreted as days and the actual closing price of the stock on y-axis. The LSTM predictions follow the actual values closely, capturing the overall trend and fluctuations well. The alignment between the blue points (actual values) and the red line (predictions) is strong, indicating a good fit. There are minor deviations, but the model effectively captures both upward and downward trends, including peaks and troughs.

Prediction on the test data 10 years SVR:

The Figure 29 depicts that the red line (predictions) fails to capture the overall trend of the blue points (actual values) accurately. There are considerable gaps between the actual values and predictions, especially during periods of significant change in the data. Additionally, the prediction line diverges significantly, often moving in the opposite direction of the actual values, indicating a substantial failure to model the underlying patterns in the data.

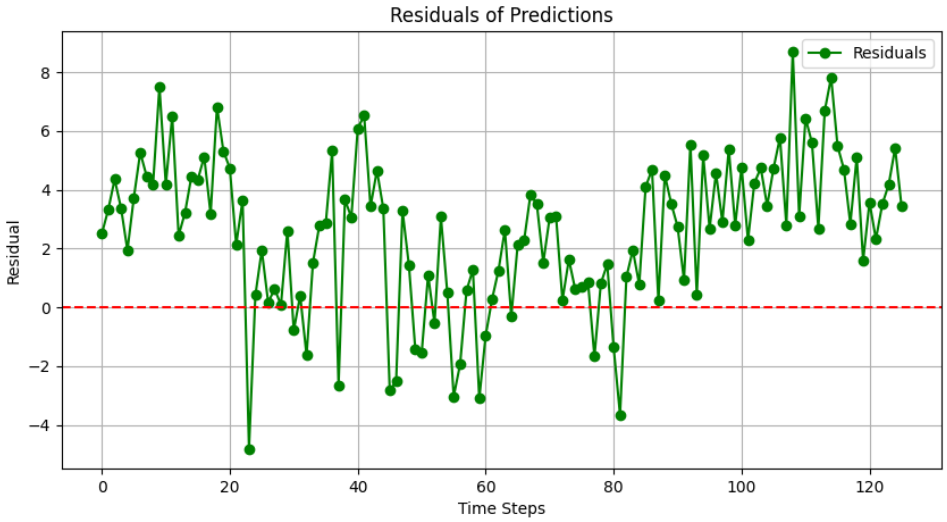

Figure 30: The plot displays the residuals for the LSTM model on the test data over a period of 5 years

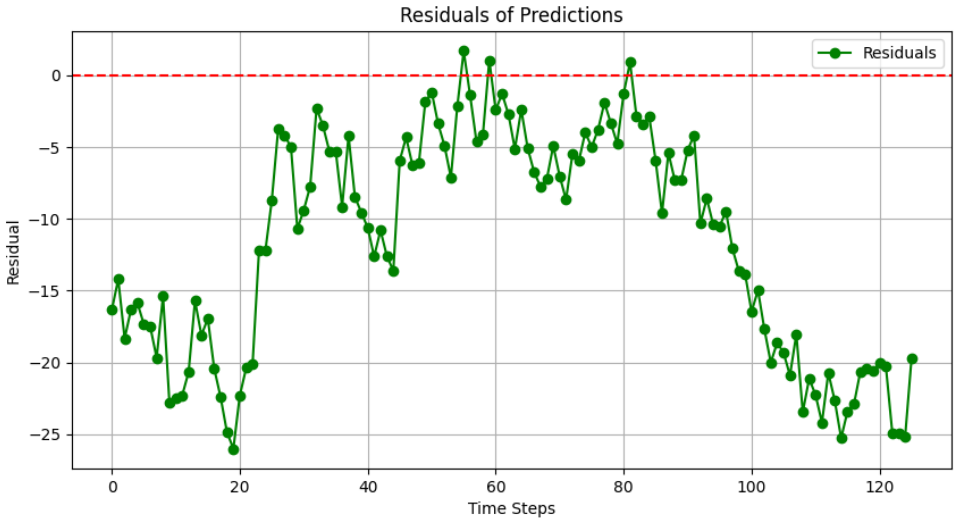

Figure 31: The plot displays the residuals for the SVR model on the test data over a period of 5 years

Residuals of predictions 5 years LSTM:

The Figure 30 depicts the residual plot for LSTM. Residuals are the vectoral differences between predictions and actual values, e.g. positive residual means overprediction. The x-axis represents time steps, can be interpreted as days and the y-axis represents the residual value, ranging from approximately -4 to 8. Green line and points represent the residuals. Red dashed line represents the zero residual line, perfect prediction.

The residuals fluctuate around the zero line, indicating that the predictions are generally close to the actual values. The variability in the residuals, encompassing both positive and negative deviations, suggests that the model's predictions do not exhibit a strong bias towards overestimation or underestimation. This balanced distribution of residuals implies that the model has effectively learned the underlying patterns in the data and is capable of making accurate predictions without systematic bias.

Residuals of predictions 5 years SVR:

The Figure 31 depicts the residual plot for SVR. The model's residual value, ranging from approximately -25 to 0. The residuals are dominantly negative, indicating that the model tends to underpredict the actual values. This occurs because, in the context of the stock's 5-year data distribution, there is an evident increasing trend in the more recent years. When the data is divided into training, validation, and test sets, the training set primarily consists of older trends in the distribution. The sharp increasing trend is predominantly present in the more recent years, which corresponds to the test set. Since the test set includes the most recent portion of the data, the model, having been trained mostly on older data, is not accustomed to this recent increasing trend. Consequently, when the test set is introduced, the model fails to recognize and predict this sharp upward trend accurately, leading to systematic underpredictions.

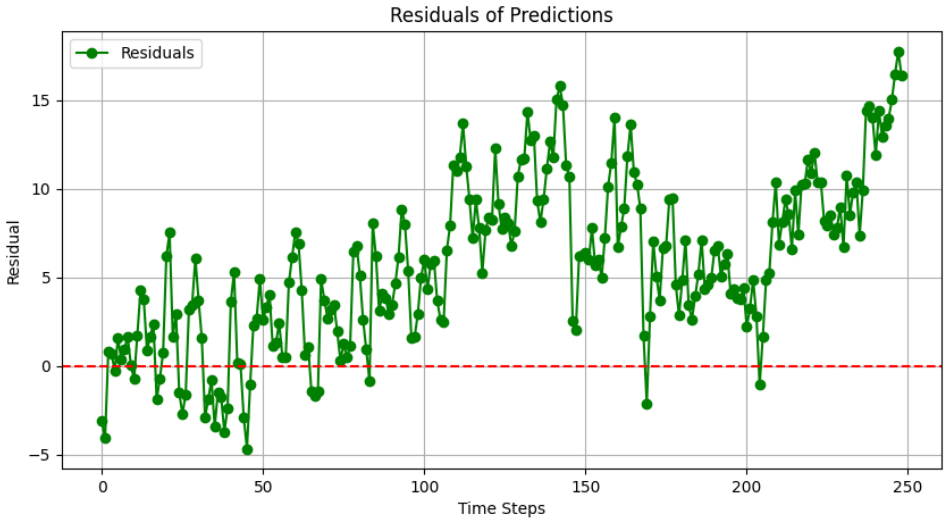

Figure 32: The plot displays the residuals for the LSTM model on the test data over a period of 10 years

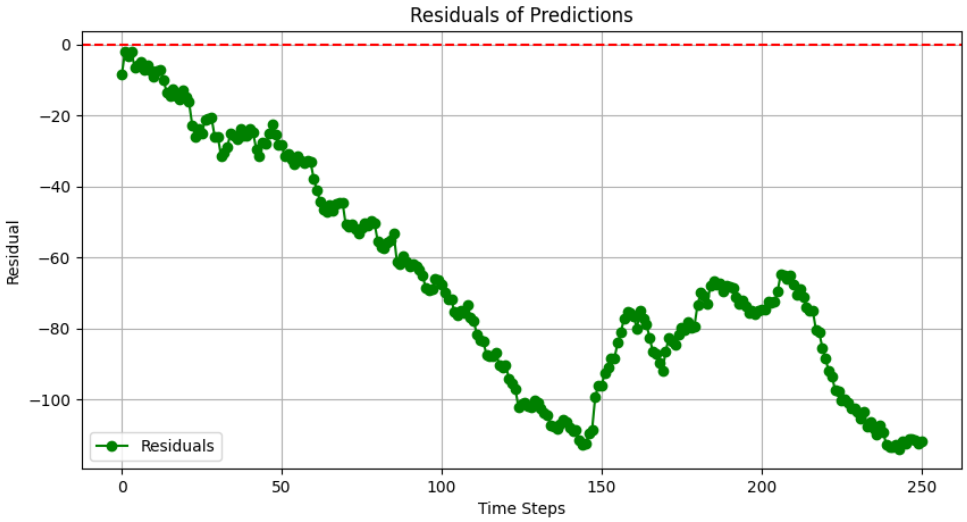

Figure 33: The plot displays the residuals for the SVR model on the test data over a period of 10 years

Residuals of predictions 10 years LSTM:

The Figure 32 depicts the residual plot for LSTM. The residuals, ranging from approximately -5 to 15. The residuals fluctuate around the zero line, indicating that the predictions are close to the actual values. There is variability in the residuals, with both positive and negative deviations. The residuals show an increasing trend over time, with more positive deviations towards the latter part of the time steps. Despite the variability, the residuals generally stay within a range of -5 to 15.

Residuals of predictions 10 years SVR:

The Figure 33 depicts the residual plot for LSTM. The residuals, ranging from approximately -120 to 0, are all negative and far from the zero line, indicating a significant underestimation in the predictions. Similar to the 5-year residual plot, also in this dataset SVR constantly underestimates the actual values. The range is wider in this set because the test set is more exposed to the upward trend. Therefore, it is evident that SVR has limitations in adapting to recent trends not present in the training data, resulting in a persistent underestimation of the actual values.

Comparison of the models' performances:

Model	Dataset	Mean Squared Error	Root Mean Squared Error	Mean Absolute Error
SVR	5 years	195,0482	13,966	11,6949
LSTM	5 years	13,1759	3,6299	3,1105
SVR	10 years	5450,8418	73,8298	66,8261
LSTM	10 years	54,4386	7,3783	6,1424

Figure 34: Performance Metrics for SVR and LSTM Models on 5-Year and 10-Year Datasets

This table presents the performance metrics, including Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE), for the SVR and LSTM models when applied to 5-year and 10-year datasets. The results clearly indicate that LSTM outperformed SVR in both datasets, with SVR failing to capture the trends and exhibiting relatively high margin errors.

7 Discussion

The experiments demonstrate that the LSTM model outperformed the SVR model on both datasets. The LSTM's ability to effectively learn from the data allows it to perform well even when encountering unexpected patterns. In contrast, the SVR model struggles with such patterns, as evidenced by the prediction and residual plots.

The initial motivation for comparing LSTM and SVR stemmed from the simpler learning structure of SVR. The aim was to assess whether SVR could perform comparably to

LSTM on shorter datasets. Despite its simplicity, the study sought to determine if SVR could approach the performance of LSTM overall. However, the experimental results indicate that SVR fails to capture intricate details in the data, such as unexpected patterns and sharp fluctuations, even in shorter datasets. The more complex, sequence-based architecture of LSTM makes it superior in all experiments conducted in this study.

The residual plots provide clear evidence of the differences between LSTM and SVR. When examining the actual data's increasing trends and comparing them with the SVR's prediction plot, it is apparent that LSTM can better predict changes in price trends using the existing features in the data.

Another noteworthy observation is that both LSTM and SVR performed better on the 5-year dataset compared to the 10-year dataset. At first glance, one might expect that more data would lead to better performance for both models, particularly for LSTM. However, the results suggest otherwise. This can be attributed to several factors:

Although the 10-year dataset contains more information, this additional data also makes the test set more challenging to predict. The increased complexity and variability in the longer dataset may have introduced unique dynamics that are harder for the models to learn.

The extra 5 years in the dataset may not simply represent a larger version of the 5-year dataset but rather include unique and significantly different dynamics. These additional complexities likely created more difficult patterns for the models to learn and generalize from.

A significant observation is that the sharp increasing trend is predominantly present in the more recent years, which corresponds to the test set. Since the test set includes the most recent portion of the data, SVR, having been trained mostly on older data, is not accustomed to this recent increasing trend. Consequently, when the test set is introduced, SVR fails to recognize and predict this sharp upward trend accurately, leading to systematic underpredictions. This highlights a limitation in SVR's ability to adapt to sudden changes in trend, underscoring the need for further refinement and potential incorporation of more recent data in the training set to improve prediction accuracy.

The experiments highlight the LSTM model's superior ability to handle complex, sequential data compared to the SVR model. The findings also underscore the challenges posed by longer and more varied datasets, suggesting that model performance can be influenced by the nature and variability of the data, not just its quantity.

The length of the data history significantly impacts the performance of each model. For the LSTM model, the longer 10-year dataset introduced more complex and varied patterns. This complexity, while challenging, allowed the LSTM to leverage its strength in capturing intricate temporal dependencies, ultimately showcasing its robustness and adaptability. However, this same complexity proved to be an obstacle for the SVR model, which struggled to adapt to the variability, resulting in less accurate predictions. Con-

versely, the 5-year dataset, with its more consistent patterns, allowed both models to perform better, though LSTM still outperformed SVR.

The optimal parameters identified through hyperparameter tuning enabled both models to achieve their best performance. While these parameters were already detailed in Section 5.3.3, it is important to note that the LSTM model's configuration allowed it to effectively learn and generalize from the data, maintaining low error metrics. In contrast, even with optimal parameters, the SVR model did not perform as well, particularly with the more complex and extensive 10-year dataset.

7.1 Limitations

Computational Constraints: Limited computational resources could have restricted the extent of hyperparameter tuning and model training iterations, potentially affecting the final performance. Especially for the LSTM model, which is computationally exhaustive due to the number of epochs and complexity of the training process. The power of the computer used for this experiment was not sufficient enough to process the needed experiments optimally, potentially limiting the model's performance and the ability to conduct more extensive tuning and validation.

This study focused on predicting the stock price of a single stock. This limitation means that the findings may not be generalizable to other stocks or broader financial markets. Evaluating multiple stocks could provide more robust evidence of the models' predictive capabilities and ensure that the observed performance is consistent across different datasets.

8 Conclusion and Future Work

The comprehensive analysis of the plots and error metrics clearly indicates that the LSTM model outperforms the SVR model in predicting time series data. The LSTM model's ability to capture temporal patterns, maintain low error metrics, and generalize well across different dataset sizes makes it a superior choice for time series forecasting. The SVR model, while capable of capturing general trends, fails to provide the same level of accuracy and robustness, particularly with larger and more complex datasets. This analysis underscores the importance of selecting appropriate models for time series forecasting, with the LSTM model proving to be a more effective and reliable choice in this context.

Despite the LSTM showing slightly lower error metrics with the 5 years dataset, it generally performs better with larger datasets due to its capability to learn more comprehensive patterns from more data. The increased complexity and variability in the 10 years dataset present a greater challenge, but the LSTM still demonstrates robustness and superior performance compared to the SVR model.

Future work could include the addition of feature selection and engineering processes such as the addition of technical indicators, as the current experiment only utilized basic features of the stock. Incorporating more sophisticated feature engineering may help capture additional underlying patterns and improve model accuracy.

Additionally, expanding the scope of the study to include multiple stocks would provide more robust evidence of the models' predictive capabilities. This would ensure that the observed performance is consistent across different datasets and enhance the generalizability of the results. By evaluating multiple stocks, future research could verify if the models predict in a similar manner as they did with a single stock, thereby providing a more comprehensive understanding of the models' performance in various market conditions.

To extend this work to multiple stocks, several considerations and steps would be necessary. A broader dataset comprising stocks from various sectors and industries would need to be collected. This data should be preprocessed to ensure consistency and comparability across different stocks. Conducting the experiments independently would lead to significantly lengthier preprocessing and experimental procedures. However, mixing stock data introduces also additional challenges that need to be addressed:

Feature selection and engineering would become more complex, as sector-specific indicators might be relevant for some stocks but not others. Developing a robust feature engineering pipeline that can adapt to different stock characteristics would be crucial. The models would need to be evaluated not only on their overall performance but also on their ability to generalize across different market conditions and sectors. This might involve sector-specific tuning of hyperparameters and the use of ensemble methods to capture diverse market behaviors. Extending the analysis to multiple stocks would require a more comprehensive validation approach, such as cross-validation across different sectors, to ensure the models' robustness and reliability. This expansion would provide a deeper and more nuanced understanding of the models' capabilities and limitations, ultimately contributing to more effective and generalized time series forecasting models.

Future work could also incorporate external data sources such as economic indicators, news sentiment analysis, and geopolitical events. These external factors often significantly impact stock prices, and incorporating them could enhance the model's predictive accuracy. Exploring more advanced neural network architectures, such as Transformer models, which have shown promise in capturing long-term dependencies in time series data, could provide further improvements in predictive performance. Developing real-time prediction capabilities would be beneficial for practical applications. This would involve optimizing the models for low-latency predictions and integrating them with real-time data feeds.

Additionally, investigating the robustness of the models to sudden market shocks, such as financial crises or significant market events, could provide insights into their reliability under extreme conditions. This could involve stress testing the models with historical market shock data. Implementing ensemble methods that combine multiple models (e.g., combining LSTM with SVR and other models) might improve prediction accuracy and robustness by leveraging the strengths of different approaches.

References

[1] Burton G. Malkiel. The efficient market hypothesis and its critics. In *Journal of Economic Perspectives*, volume 17(1), pages 59–82, 2003.

[2] Adam Hayes. Financial markets: Role in the economy, importance, types, and examples. In *Investopedia*, 2023.

[3] Jonathan Berk and Peter DeMarzo. Corporate finance, global edition. In *Corporate Finance, Global Edition*, page 44, 2019.

[4] Miquel Noguer I Alonso. Economy and financial markets as complex systems modeling the economy: A new framework. In *Social Science Research Network*, 2024.

[5] Marcel Arsenault, Jim Clayton, and Liang Peng. Mortgage fund flows, capital appreciation, and real estate cycles. In *The Journal of Real Estate Finance and Economics*, volume 47(2), pages 243–265, 2013.

[6] Barclay Palmer. Institutional investors vs. retail investors: What's the difference? In *Investopedia*, 2023.

[7] James Chen. Risk-return tradeoff: How the investment principle works. In *Investopedia*, 2023.

[8] Eugene F. Fama. The behavior of stock-market prices. In *The Journal of Business*, volume 38, 1965.

[9] Twssadmin. Predicting stock prices with machine learning and financial modelling. In *The Wall Street School*, 2024.

[10] StockRaven.com. How to forecast stock prices using ml and dl algorithms. In *Stock Raven*, 2024.

[11] Xue Ying. An overview of overfitting and its solutions. In *Journal of Physics: Conference Series*, volume 1168, page 022022, 2019.

[12] FSE Editors and Writers. Time series analysis: Understanding temporal trends and patterns. 2023.

[13] Jasdeep Bhatia. Time series analysis in data science: A step-by-step guide. In *Learn Coding Anywhere Anytime -PW Skills Blog*, 2024.

[14] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. In *European Journal of Operational Research*, volume 270, pages 654–669, 2018.

[15] Adil Moghar and Mhamed Hamiche. Stock market prediction using LSTM recurrent neural network. In *The 11th International Conference on Ambient Systems, Networks and Technologies (ANT 2020) / The 3rd International Conference on Emerging Data and Industry 4.0 (EDI40 2020) / Affiliated Workshops, April 6-9, 2020, Warsaw, Poland*, volume 170 of *Procedia Computer Science*, pages 1168–1173, 2020.

[16] Xiaochun Zhang, Chen Li, Kuan-Lin Chen, Dimitrios Chrysostomou, and Hongji Yang. Stock prediction with stacked-lstm neural networks. In *21st IEEE International Conference on Software Quality, Reliability and Security, QRS 2021 - Companion, Hainan, China, December 6-10, 2021*, pages 1119–1125, 2021.

[17] Kyoung-Jae Kim. Financial time series forecasting using support vector machines. In *Neurocomputing*, volume 55, pages 307–319, 2003.

[18] Jingdong Yang. Support vector machine-based stock prediction analysis. In *Highlights in Business, Economics and Management*, volume 3, pages 12–18, 2023.

[19] Latrisha N. Mintarya, Jeta N.M. Halim, Callista Angie, Said Achmad, and Aditya Kurniawan. Machine learning approaches in stock market prediction: A systematic literature review. In *Procedia Computer Science*, volume 216, pages 96–102, 2023.

[20] Lijuan Cao and Francis Eng Hock Tay. Financial forecasting using support vector machines. In *Neural Comput. Appl.*, volume 10, pages 184–192, 2001.

[21] Jigar Patel, Sahil Shah, Priyank Thakkar, and Ketan Kotecha. Predicting stock market index using fusion of machine learning techniques. In *Expert Syst. Appl.*, volume 42, pages 2162–2172, 2015.

[22] Vikram Sharma, Manik Rakhra, and Gauri Mathur. Hybrid approaches for stocks prediction and recommendation system. In *E3S Web of Conferences*, volume 453, 2023.

[23] Wen Long, Jing Gao, Kehan Bai, and Zhichen Lu. A hybrid model for stock price prediction based on multi-view heterogeneous data. In *Financial Innovation*, volume 10, page 48, 2024.

[24] Brahmanapalli Kalyan, S Reddy, Krovvidi Kumari, and Manish Jain. Comparative analysis of stock price prediction accuracy: A machine learning approach with arima, lstm, and random forest models. In *International Research Journal on Advanced Engineering Hub (IRJAEH)*, volume 2, pages 1141–1151, 2024.

[25] Wei Huang, Yoshiteru Nakamori, and Shouyang Wang. Forecasting stock market movement direction with support vector machine. In *Comput. Oper. Res.*, volume 32, pages 2513–2522, 2005.

[26] Indronil Bhattacharjee and Pryonti Bhattacharja. Stock price prediction: A comparative study between traditional statistical approach and machine learning approach. In *2019 4th International Conference on Electrical Information and Communication Technology (EICT)*, pages 1–6, 2019.

[27] Mahfuz Islam Khan Jabed. Stock market price prediction using machine learning techniques. In *American International Journal of Sciences and Engineering Research*, volume 7, pages 1–6, 2024.

[28] Peter Bloem. Machine learning lecture slides. In *Vrije Universiteit Amsterdam*. MLVU, 2018.

[29] Tasmay Pankaj Tibrewal. Support vector machines (svm): an intuitive explanation. In *Medium*, 2024.

[30] Nitin Nagar, Pradeep K Jatav, Mahak Gupta, and Ankit Limone. Performance comparison of lstm and svr models in predicting stock prices. In *Journal of Harbin Engineering University*, volume 7, pages 1006–7043, 2023.

[31] Abhishek Kothari, Atharv Kulkarni, Tejas Kohade, and Chetan Pawar. Stock market prediction using lstm. In *Lecture notes in networks and systems*, pages 143–164, 2024.

[32] Siddhant Behera, Chandra Prakash, and Narendra Sharma. A combined model for index price forecasting using lstm, rnn, and gru. In *Lecture notes in networks and systems*, pages 499–514, 2024.

[33] S Dinesh, S Rahul, O Naga Sandeep, and Adduri Maruthi Siva Rama Raju. Stock price prediction using lstm. In *ResearchGate*, 2021.

[34] Mingshi Wang. Attention lstm: application in stock price prediction on single companies. In *TU Delft Repositories*, 2021.

[35] Janki Patel, Prof. Miral Patel, and Prof. Mittal Darji. Stock price prediction using rnn and lstm. In *Journal of Emerging Technologies and Innovative Research*, volume 5(11), pages 1069–1071, 2018.

[36] Ran Aroussi. yfinance: Download market data from yahoo! finance api. 2024.

[37] J. D. Hunter. Matplotlib: A 2d graphics environment. In *Computing in Science & Engineering*, volume 9(3), pages 90–95. IEEE Computer Soc, 2007.

[38] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. In *Nature*, volume 585, pages 357–362. Springer Science and Business Media LLC, 2020.

[39] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017.

[40] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. In *Journal of machine learning research*, volume 12, pages 2825–2830, 2011.

[41] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *arXiv.org*, 2014.